چالش ۳ از ۳ — «تصمیم پرهزینه»

مرحله: سوم (پیش‌نیاز: Database مرحله ۲ — پیش‌نیاز چالش نهایی)

محور: مقدمه ML، انواع یادگیری، ساختار Dataset، Data Preprocessing، Feature Engineering، Classification، Classification Evaluation

زمان پیشنهادی: ۶ تا ۸ روز کاری

سقف زمان اجرای نوتبوک: ۲۰ دقیقه

شرایط تحویل در Delivery_Guide_۰۰ است.

صورت مسئله در یک نگاه

یک مؤسسه اعتباری می‌خواهد پیش از اعطای تسهیلات، احتمال نکول جدی متقاضی را تخمین بزند. داده تاریخی در دست است: برای هر متقاضی، وضعیت درآمد، نسبت بدهی، تعداد خطوط اعتباری، سن، تعداد افراد تحت تکفل، و سابقه تأخیرهای پرداخت — و این‌که در دو سال بعد دچار نکول جدی شده یا نه.

اما مسئله «ساختن یک مدل با دقت بالا» نیست. مسئله این است:

دو نوع اشتباه در این سیستم هزینه برابر ندارند.

- اگر مدل بگوید فردی سالم است و او نکول کند (False Negative)، مؤسسه کل مبلغ را از دست می‌دهد.
- اگر مدل بگوید فردی پرخطر است و در واقع سالم باشد (False Positive)، مؤسسه فقط سود یک وام را از دست می‌دهد.

ماتریس هزینه تعیین‌شده برای این چالش:

	پیش‌بینی: سالم	پیش‌بینی: پرخطر
---	---	---
واقعیت: سالم	۰ واحد	۱ واحد
واقعیت: نکول	۱۲ واحد	۰ واحد

مسئله شما: مدلی بسازید و آن را طوری تنظیم کنید که هزینه کل تصمیم‌ها را کمینه کند — و ثابت کنید که این عدد روی داده دیده‌نشده پایدار است.

و یک هشدار صریح: در این چالش، Accuracy معیار تصمیم‌گیری نیست. داده به‌شدت نامتوازن است و در چنین داده‌ای، Accuracy بالا می‌تواند کاملاً بی‌معنا باشد. چرا و چقدر بی‌معنا، چیزی است که خودتان باید اندازه بگیرید و در گزارش نشان دهید. اگر Accuracy دلیل انتخاب یک مدل باشد، آن بخش نمره نمی‌گیرد.

منبع داده

Give Me Some Credit — رقابت عمومی Kaggle

- GiveMeSomeCredit :Kaggle (فایل cs-training.csv)
- توجه: این یک *competition* است نه dataset؛ برای دریافتش باید قواعد رقابت را بپذیرید و از مسیر دریافت مخصوص competition استفاده کنید (برخلاف Olist در مرحله ۲). اگر دسترسی به Kaggle برایتان مشکل داشت، پیش از شروع اعلام کنید.
- حدود ۱۵۰٬۰۰۰ رکورد، ۱۰ ویژگی ورودی، یک برچسب دودویی.
- نسبت کلاس مثبت حدود ۷٪ — یعنی داده نامتوازن است.
- فایل راهنمای دیکشنری داده همراه آن منتشر شده است؛ خواندن آن اجباری است. چند ستون معنایی دارند که از نام‌شان پیدا نیست.

الزام زنجیره: داده از Database می‌آید، نه از CSV

- این dataset باید با همان package مرحله ۱ و در همان Database مرحله ۲ بارگذاری شود.
- این منبع یک فایل است، نه API. پس کارخانه مرحله ۱ باید بتواند منبع تازه‌ای از جنس دیگر را بپذیرد و همان اعتبارسنجی، قرنطینه و manifest را رویش اعمال کند. اگر معماری فعلی‌تان این را نمی‌پذیرد، بازسازی‌اش بخشی از این مرحله است و وقتش را باید در برنامه‌تان حساب کنید. نوشتن یک اسکریپت بارگذاری جداگانه و کنار گذاشتن package، قبول نیست.
- نوتبوک مرحله ۳ اجازه ندارد مستقیم CSV بخواند. تمام خواندن داده باید از Database و با SQLAlchemy باشد.
- حداقل یک بخش از آماده‌سازی داده باید در لایه Database انجام شود (مثلاً از طریق یک View) و در گزارش بگویید چرا آن بخش را آن‌جا گذاشتید و بقیه را در Python.

آن‌چه باید تولید شود

۱) شناخت dataset و برچسب

- ساختار داده را مشخص کنید: Target، Features، Samples، و این‌که هر ویژگی عددی است یا دسته‌ای. ستون‌هایی که عددی به نظر می‌رسند اما در واقع دسته‌ای‌اند باید شناسایی شوند.

- مقادیر نگهبان (sentinel): در این داده بعضی ستون‌ها مقادیری دارند که عدد واقعی نیستند بلکه «کد» هستند. پیدا کردن‌شان و تعیین تکلیف‌شان بخشی از مسئله است. اگر آن‌ها را عدد معمولی فرض کنید، مدل‌تان روی داده واقعی خراب می‌شود.
- Outlierهای غیرممکن: داده مقادیری دارد که از نظر دامنه معنادار نیستند. سیاست شما در برابرشان باید روشن و توجیه‌شده باشد.
- در گزارش بنویسید این مسئله در کدام دسته یادگیری قرار می‌گیرد و چرا؛ و اگر همین داده را به مسئله Regression تبدیل می‌کردیم، چه چیزی عوض می‌شد.

۲) گم‌شده‌ها (پیچیدگی اضافه‌شده)

این داده مقادیر گم‌شده دارد. سؤال اول این نیست که «چطور پرشان کنیم»، بلکه این است که نبودِ مقدار خودش معنا دارد یا نه.

- ابتدا بررسی کنید آیا الگوی گم‌شدگی تصادفی است یا نه — و پاسخ‌تان را با شاهد عددی بدهید، نه با فرض.
- پاسخ این بررسی باید روی سیاست شما اثر بگذارد. اگر اثر نگذاشت، بگویید چرا.
- حداقل دو سیاست متفاوت برای مواجهه با گم‌شده‌ها پیاده و با عدد مقایسه کنید.
- در گزارش بگویید کدام را انتخاب کردید و چه چیزی را به‌دست یا از دست دادید.

۳) آماده‌سازی داده با مرز نشت‌ناپذیر

الزام سخت: هیچ آماری از داده ارزیابی نباید در آماده‌سازی داده آموزش استفاده شود — نه میانگین، نه انحراف معیار، نه دامنه مقادیر، نه فهرست دسته‌ها.

- باید نشان دهید که این مرز رعایت شده است. ادعای شفاهی کافی نیست؛ باید در نوتبوک قابل بازرسی باشد.
- آماده‌سازی باید به شکلی باشد که بدون تغییر کد، عیناً روی یک فایل داده تازه با همان schema قابل اعمال باشد.

شامل: تعیین تکلیف گم‌شده‌ها، Encoding متغیرهای دسته‌ای، و Scaling. برای هر مدلی که آموزش می‌دهید، لازم بودن یا نبودن Scaling باید جداگانه بررسی و توجیه شود — با ارجاع به این‌که آن مدل چطور کار می‌کند.

۴) Feature Engineering با ممیزی نشت

- حداقل ۴ ویژگی جدید بسازید که از ترکیب یا تبدیل ستون‌های موجود به دست آمده باشند و منطق کسب‌وکاری داشته باشند. برای هرکدام یک جمله: چه فرضیه‌ای را آزمایش می‌کند؟
- اثر هر ویژگی را بسنجید: با و بدون آن، متریک چه تغییری می‌کند؟ ویژگی‌ای که چیزی اضافه نکند باید حذف شود و حذفش گزارش شود.
- بخش اجباری «ممیزی نشت داده»: برای هر ویژگی ساخته‌شده، توضیح دهید که آیا در لحظه تصمیم‌گیری واقعی (یعنی *قبل* از اعطای وام) این اطلاع در دسترس است یا نه. ویژگی‌ای که از آینده اطلاع می‌گیرد، هر چقدر هم متریک را بالا ببرد، تقلب است و باید حذف شود. اگر چنین ویژگی‌ای ساختید و بعد کشفش کردید، همین ماجرا را در گزارش بنویسید — این یکی از ارزشمندترین بخش‌های گزارش است.

۵) خط مبنا، و مدل‌هایی که باید از آن بهتر باشند

- یک خط مبنای ساده بسازید: چیزی بدون یادگیری، یا با یک قاعده دستی روی یک ستون. عدد هزینه آن را گزارش کنید. این عدد، سطح زمین است.
- حداقل دو خانواده مدل متفاوت از scikit-learn آموزش دهید (نه دو تنظیم از یک مدل).
- شرط پذیرش: بهترین مدل شما باید هزینه خط مبنا را حداقل ۲۵٪ کاهش دهد. اگر نرسید، باید در گزارش توضیح دهید چرا — و توضیح پذیرفتنی هم ممکن است.

۶) انتخاب آستانه از روی هزینه، نه از روی عدد پیش‌فرض

این مهم‌ترین بخش این چالش است:

- خروجی مدل یک احتمال است، اما آن‌چه مؤسسه اجرا می‌کند یک تصمیم دودویی است. نقطه‌ای که احتمال را به تصمیم تبدیل می‌کند، مقدار پیش‌فرض کتابخانه نیست و باید انتخاب شود.
- این انتخاب باید با ماتریس هزینه بالا و با عدد توجیه شود، نه با قاعده سرانگشتی.
- نتیجه نهایی را به‌صورت هزینه مورد انتظار به ازای هر ۱۰۰۰ متقاضی گزارش کنید. این عدد، زبان مشترک شما با مؤسسه است.
- نشان دهید اگر ماتریس هزینه عوض شود (مثلاً نسبت ۱۲ به ۱ بشود ۴ به ۱)، آستانه بهینه چه می‌شود. یک جمله در مورد معنای این حساسیت.

۷) ارزیابی کامل و پایداری

برای بهترین مدل:

- Confusion Matrix با اعداد مطلق، در آستانه انتخابی خودتان.
- Precision، Recall، F۱، AUC — و در گزارش، برای «هر» متریک یک جمله که می‌گوید در این مسئله چه چیزی را می‌سنجد و چرا به‌تنهایی کافی نیست.
- Accuracy را هم گزارش کنید، فقط برای این‌که نشان دهید چرا گمراه‌کننده است.
- پایداری (پیچیدگی اضافه‌شده): نتیجه یک تقسیم تک‌مرحله‌ای پذیرفته نیست. ارزیابی باید چندتکه‌ای باشد و پراکندگی نتیجه گزارش شود. مدلی که متریکش بین تکه‌ها نوسان زیاد دارد، قابل اتکا نیست — و اگر چنین است باید بگویید.
- یک مجموعه نهایی را از ابتدا کنار بگذارید و فقط یک بار، در پایان، به آن نگاه کنید. اگر عدد نهایی با عدد اعتبارسنجی فاصله معنادار داشت، همین فاصله باید در گزارش تحلیل شود.

۸) تفسیر مدل و شناسنامه مدل

- پنج ویژگی مؤثرتر را استخراج و گزارش کنید. آیا با شهود کسب‌وکاری هم‌خوان است؟ اگر نه، آن ناهم‌خوانی را بررسی کنید؛ معمولاً یا یافته جالبی است یا اشکالی در داده.

- یک Model Card کوتاه (نیم تا یک صفحه): مدل چه کاری می‌کند، روی چه داده‌ای آموزش دیده، چه چیزی را نمی‌تواند، و برای چه جمعیتی نباید استفاده شود.
- یک بخش «کجا می‌شکند؟»: سه موقعیت واقعی که این مدل در آن‌ها تصمیم بد می‌گیرد.
- یک پاراگراف درباره انصاف: آیا متریک‌ها در زیرگروه‌های مختلف (مثلاً گروه‌های سنی) یکسان است؟ اگر نه، معنایش برای مؤسسه چیست؟

۹) مهندسی، تست و Git

- کد مدل باید در `/src` باشد و تکرارپذیر؛ نوتبوک روایت‌گر است.
- مدل آموزش‌دیده و تنظیمات آماده‌سازی باید ذخیره شوند و یک تابع/اسکریپت وجود داشته باشد که یک فایل تازه را با همان schema بگیرد و خروجی پیش‌بینی بدهد.
- حداقل ۸ تست: درستی توابع ساخت ویژگی، رفتار آماده‌سازی در حضور مقدار گم‌شده، درستی محاسبه تابع هزینه، و این‌که مسیر پیش‌بینی روی یک ورودی نمونه کار می‌کند.
- حداقل ۱۰ commit معنادار.

معیار پذیرش این مرحله

- ☐ داده از Database خوانده شود، نه از CSV.
- ☐ مرز نشت‌ناپذیر بین آموزش و ارزیابی قابل بازرسی باشد.
- ☐ خط مبنا وجود داشته باشد و هزینه‌اش گزارش شده باشد.
- ☐ حداقل دو خانواده مدل مقایسه شده باشند.
- ☐ آستانه تصمیم با ماتریس هزینه توجیه شده باشد، نه ۰٫۵ پیش‌فرض.
- ☐ Confusion Matrix، Precision، Recall، F۱ و AUC گزارش شده باشند.
- ☐ ارزیابی چندتکه‌ای با پراکندگی گزارش شده باشد.
- ☐ بخش ممیزی نشت داده نوشته شده باشد.
- ☐ هزینه به ازای هر ۱۰۰۰ متقاضی به‌صورت یک عدد صریح اعلام شده باشد.

پیش‌نمایش چالش نهایی

تا این‌جا سه قطعه را جدا جدا ساخته‌اید: کارخانه داده، Database بهینه، و مدل. در چالش نهایی این سه باید یک سامانه شوند و روی یک مسئله واحد کار کنند — با محدودیت سختی که این‌جا نداشتید: زمان. آن‌جا فقط اجازه دارید از اطلاعاتی استفاده کنید که در لحظه تصمیم واقعاً موجود بوده است، و تقسیم داده بر اساس زمان انجام می‌شود نه تصادف. اگر الان عادت ممیزی نشت را در خودتان ساخته باشید، آن مرحله خیلی راحت‌تر می‌شود.